ПРАКТИЧЕСКАЯ РАБОТА № 5

Разработка модулей системы «Медицинский центр «Медика»»

МДК.04.01 Внедрение, поддержка компьютерных систем и обеспечение качества функционирования компьютерных систем

1. Цель работы

Разработать пять модулей информационной системы: пользователи и роли, каталог, клиенты, заказы и отчёты. Для каждого модуля создать модели, административный интерфейс, сериализаторы и REST-эндпоинты.

2. Схема модулей

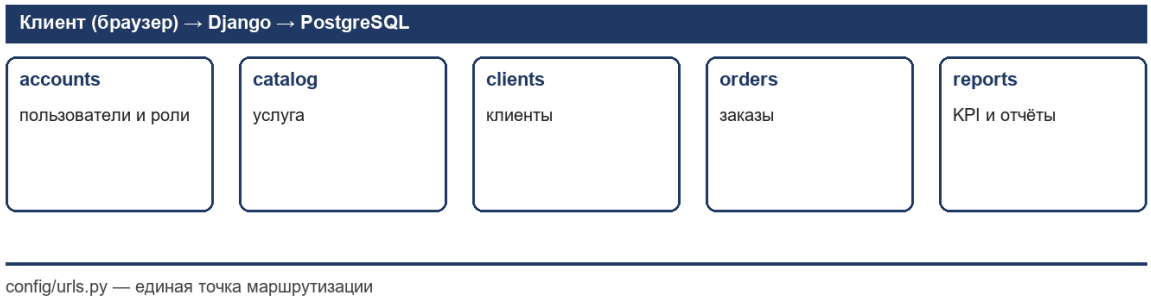


Рисунок 1 — Схема модульной структуры приложения

Модуль	Назначение	Основные объекты
accounts	пользователи и роли	User
catalog	специализация и услуга	Specialization, Service
clients	клиенты и история обращений	Client
orders	заказы и статусы	Order
reports	KPI и аналитика	—

3. Модуль accounts — пользователи и роли

Расширяем стандартную модель пользователя Django, добавляя роль и телефон. Роль определяет доступные действия в системе.

```
class User(AbstractUser):
    ROLE_CHOICES = [
        ('admin', 'Администратор'),
        ('manager', 'Менеджер'),
        ('doctor', 'Врач'),
    ]
    role = models.CharField('Роль', max_length=20,
                            choices=ROLE_CHOICES, default='manager')
    phone = models.CharField('Телефон', max_length=20, blank=True)
```

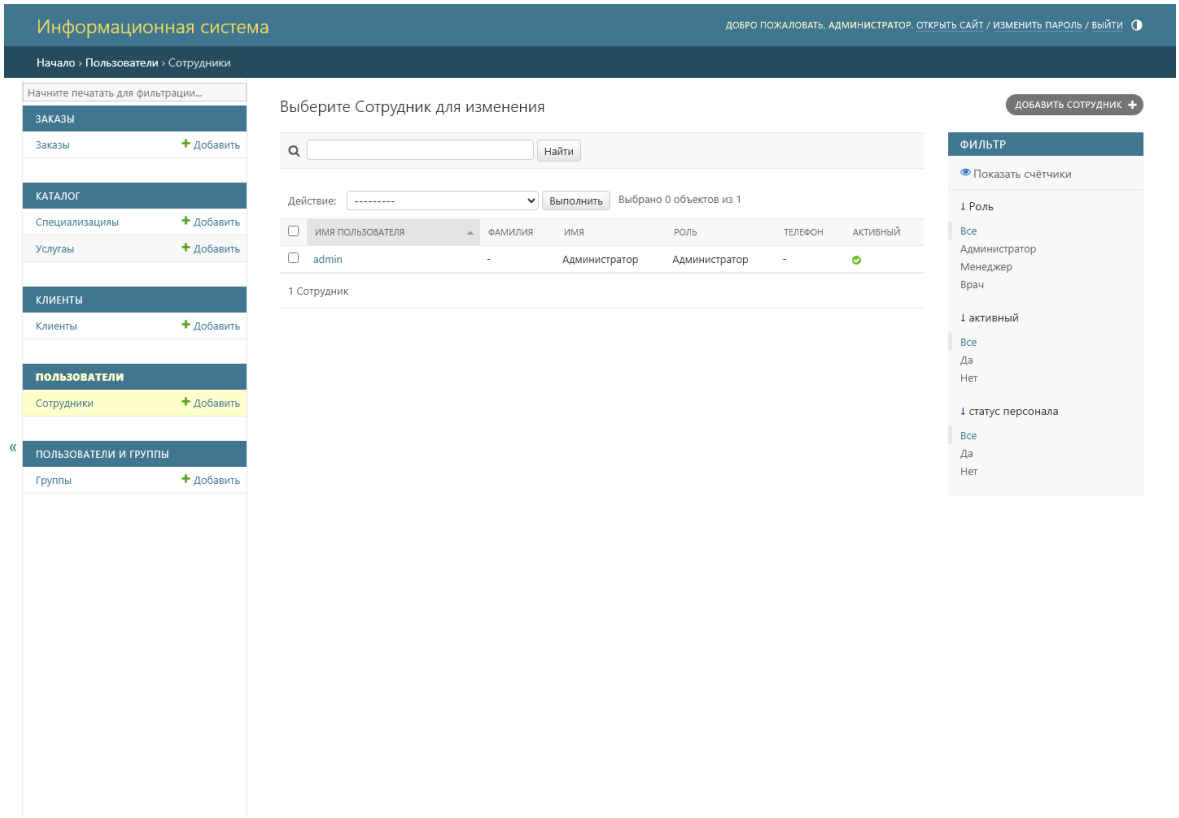


Рисунок 2 — Раздел «Сотрудники» в админке

4. Модуль catalog — специализация и услуга

Модуль хранит справочник и позиции прайса. Связь «один ко многим» реализована внешним ключом с защитой от удаления справочной записи.

```
class Specialization(models.Model):
    name = models.CharField('Название', max_length=100, unique=True)


class Service(models.Model):
    STATUS_CHOICES = [
        ('active', 'Доступна'),
        ('reserved', 'Забронирована'),
        ('archived', 'Архив'),
    ]
    specialization = models.ForeignKey(
        Specialization, on_delete=models.PROTECT, related_name='items')
    name = models.CharField('Название', max_length=200)
    price = models.DecimalField('Цена, руб.', max_digits=10, decimal_places=2)
    status = models.CharField('Статус', max_length=20,
                              choices=STATUS_CHOICES, default='active')
```

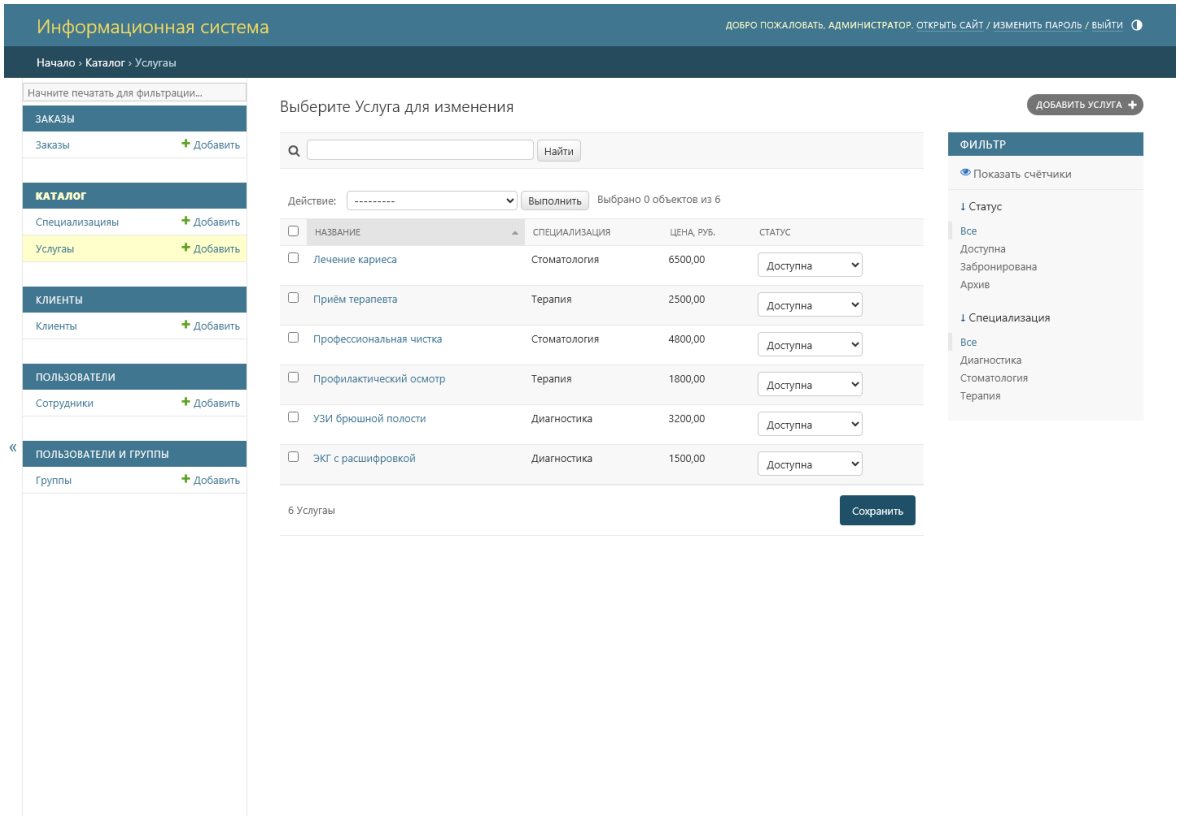


Рисунок 3 — Раздел «Услугаы» с фильтрами

5. Модуль clients — клиенты

Карточка клиента содержит контакты и дату регистрации; по ней строится история заказов.

```
class Client(models.Model):
    last_name = models.CharField('Фамилия', max_length=50)
    first_name = models.CharField('Имя', max_length=50)
    phone = models.CharField('Телефон', max_length=20)
    email = models.EmailField('Email', blank=True)
    created_at = models.DateTimeField('Дата регистрации', auto_now_add=True)

    @property
    def full_name(self):
        return f'{self.last_name} {self.first_name}'
```

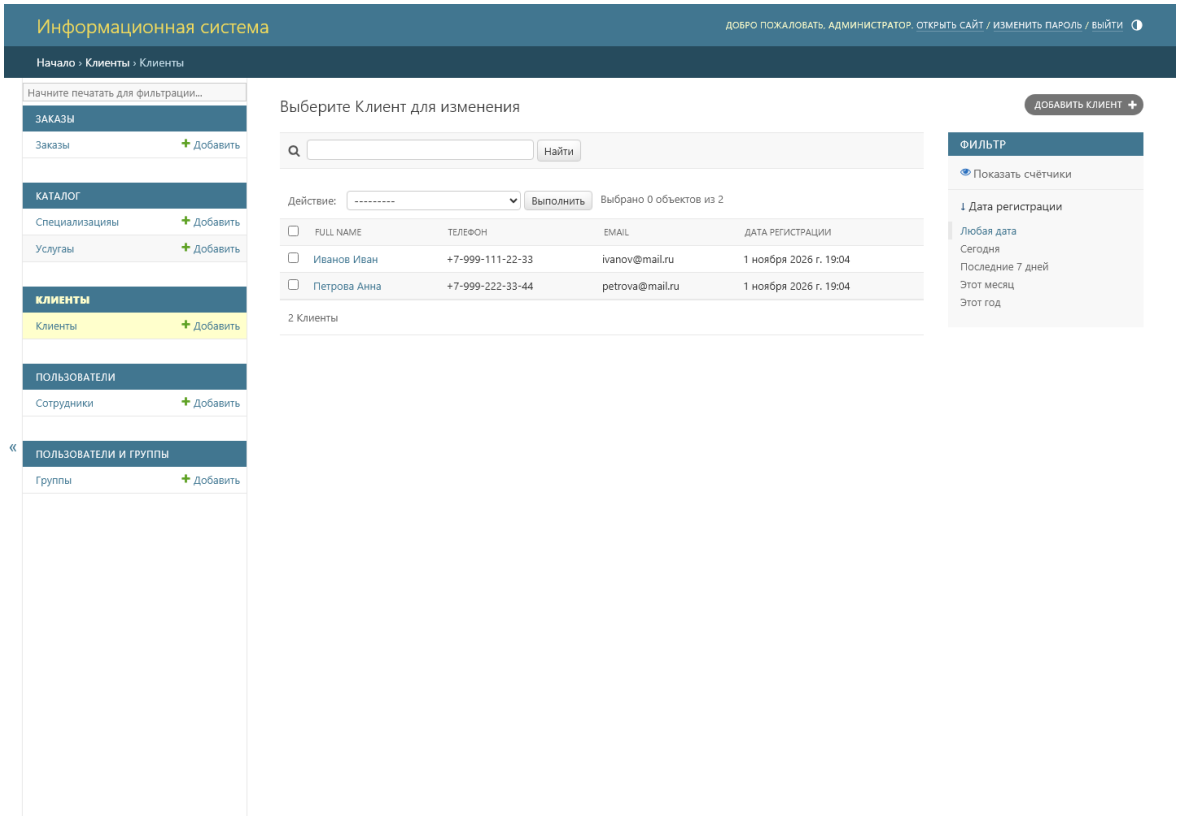


Рисунок 4 — Раздел «Клиенты»

6. Модуль orders — заказы

Ключевая бизнес-логика: при создании заказа позиция резервируется, при выполнении — уходит в архив, при отмене — возвращается в продажу. Каскад реализован в методе save() модели.

```
def save(self, *args, **kwargs):
    if not self.total:
        self.total = self.service.price
    super().save(*args, **kwargs)

    item = self.service
    if self.status in ('new', 'confirmed') and item.status == 'active':
        item.status = 'reserved'
        item.save(update_fields=['status'])
    elif self.status == 'done' and item.status == 'reserved':
        item.status = 'archived'
        item.save(update_fields=['status'])
    elif self.status == 'canceled' and item.status == 'reserved':
        item.status = 'active'
        item.save(update_fields=['status'])
```

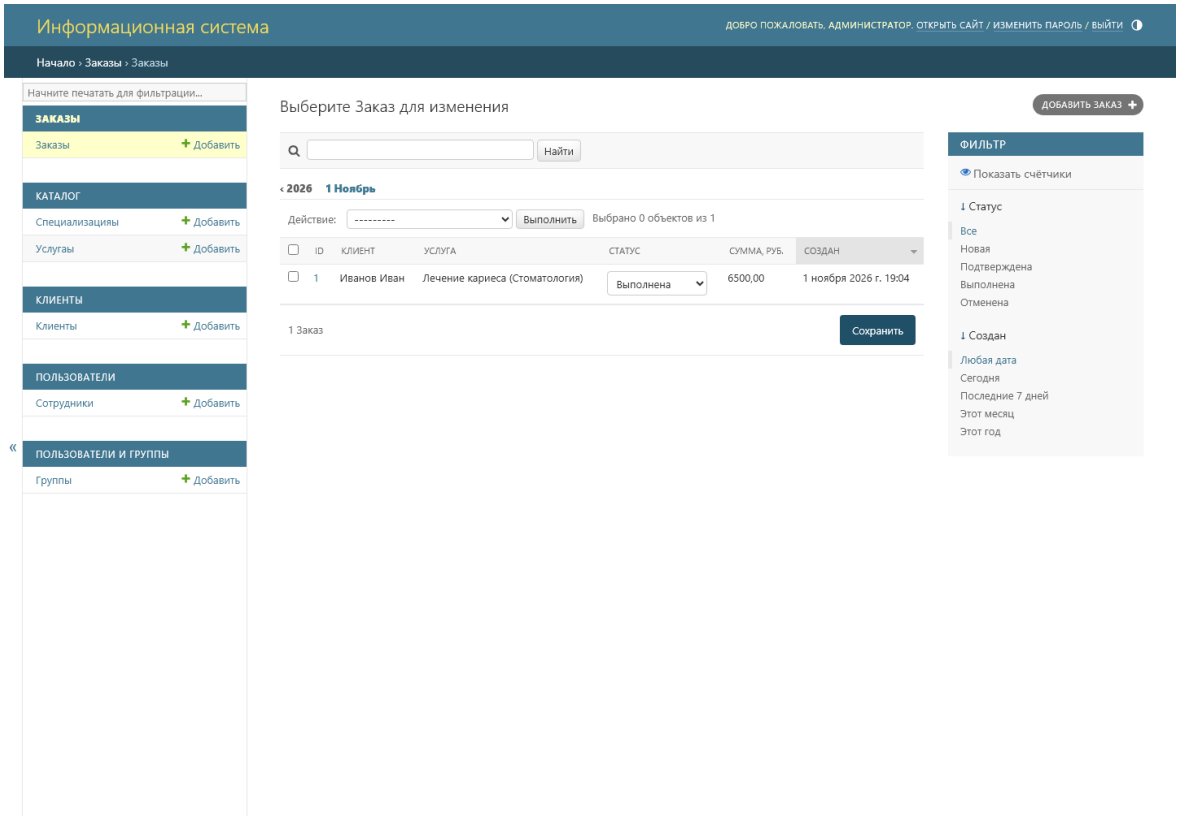


Рисунок 5 — Раздел «Заказы»

7. Модуль reports — отчёты

Модуль считает ключевые показатели одним запросом к базе и отдаёт их через API. KPI используется в дашборде и в отчёте руководителя.

```
def kpi():
    done = Order.objects.filter(status='done')
    return {
        'orders_total': Order.objects.count(),
        'orders_done': done.count(),
        'revenue': float(done.aggregate(s=Sum('total'))['s'] or 0),
        'avg_check': float(done.aggregate(a=Avg('total'))['a'] or 0),
        'top_items': list(
            Order.objects.values('service__name')
            .annotate(cnt=Count('id')).order_by('-cnt')[:5]
        ),
    }
```

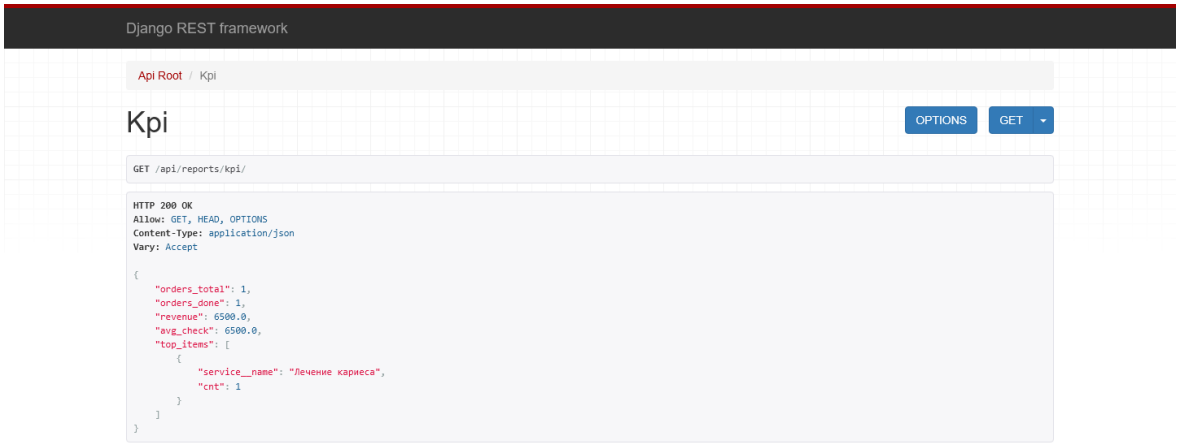


Рисунок 6 — Ответ API /api/reports/kpi/

8. Маршрутизация и API

Все модули подключаются в config/urls.py; для каждого набора данных создан ViewSet и зарегистрирован в роутере DRF.

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('apps.catalog.urls')),
    path('api/clients/', include('apps.clients.urls')),
    path('api/orders/', include('apps.orders.urls')),
    path('api/reports/', include('apps.reports.urls')),
    path('api/auth/login/', TokenObtainPairView.as_view()),
    path('api/auth/refresh/', TokenRefreshView.as_view()),
]
```

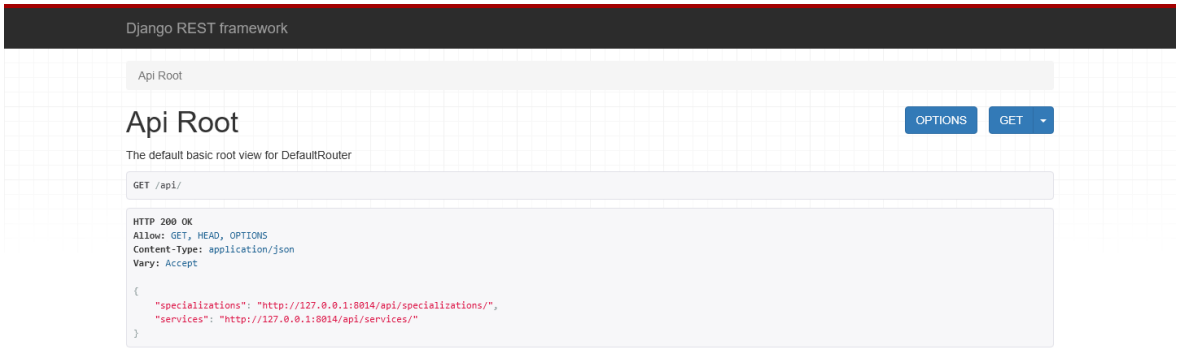


Рисунок 7 — Браузер DRF со списком эндпоинтов

9. Вывод

Разработаны пять модулей, связанных общей моделью данных. Админка позволяет вести справочники и заказы, REST API — интегрировать систему с внешними клиентами, модуль отчётов отдаёт KPI. Модульная структура упрощает развитие проекта: новый функционал добавляется отдельным приложением.